MLOps LAB ASSIGNMENT REPORT

Sessions 7, 8 & 9: ML Pipelines, FastAPI Model Serving and Docker Containerization

Student Name: Arjit Prasad Das	**Roll Number:** 24AI013
Course: B.Tech (Artificial Intelligence & Data Science)	**Lab:** MLOps — Sessions 7, 8, 9 Combined

1. Project Objectives & Scenario

The objective of this assignment is to refactor an existing Machine Learning project (House Price Prediction) into a production-grade, modular, and reproducible ML application. The project decouples data loading, preprocessing, model training, evaluation, and inference into independent modules, exposes predictions via a high-performance REST API using **FastAPI**, and packages the entire runtime into an isolated, portable **Docker container**.

2. Modular Architecture & Pipeline Design (Part A)

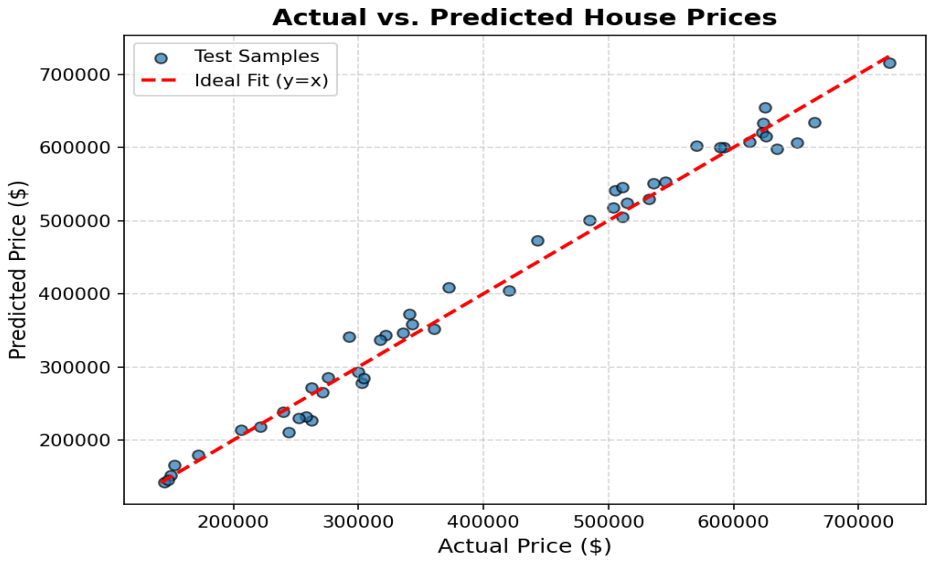
The repository follows a clean, decoupled architecture where each module has a single, well-defined responsibility. To prevent data leakage, dataset splitting is executed prior to feature scaling. Both preprocessing (StandardScaler) and estimator (RandomForestRegressor) are encapsulated into a single unified sklearn.pipeline.Pipeline and serialized together into models/house_price_pipeline.joblib, ensuring strictly identical feature transformations during training and inference.

Module	Responsibility	Key Functions / Artifacts
src/data.py	Data ingestion, schema validation, cleaning, non-leaking split	load_data(), clean_data(), split_data()
src/train.py	Reads config.yaml, constructs unified Pipeline, fits on train split	build_pipeline(), train_model(), models/*.joblib
src/evaluate.py	Scores held-out test data, computes metrics, generates plots	evaluate_model(), reports/metrics.json, figures/
src/predict.py	Loads saved pipeline, formats input DataFrame, executes inference	load_model(), predict(), sample_predictions.json
src/api.py	Exposes REST endpoints (/health, /predict) with Pydantic validation	GET /health, POST /predict, FastAPI app
Dockerfile	Packages runtime, dependencies, source code, and model artifact	Python 3.11-slim, EXPOSE 8000, Uvicorn server

3. Model Training & Held-Out Evaluation Results

Evaluation Metric	Value Achieved	Interpretation
Mean Absolute Error (MAE)	$17,227.05	Average absolute difference in predicted home price
Root Mean Squared Error (RMSE)	$21,353.52	Penalizes large prediction errors on held-out test data
Coefficient of Determination (R^2)	0.9837	Explains over 98% of target price variance
Held-out Test Samples	49	Evaluated strictly on unseen held-out samples

Figure 1: Actual vs. Predicted House Prices on Held-Out Test Set



4. FastAPI Model Serving & Schema Validation (Part B)

The model is served using **FastAPI**. The input payload is validated via pydantic.BaseModel, enforcing numerical boundaries (e.g. area_sqft > 0, bedrooms >= 1, age_years >= 0). Invalid inputs, missing fields, and type mismatches return controlled HTTP 422 Unprocessable Entity responses without server crashes.

Test Case Description	HTTP Method & Payload	Expected Result	Observed Result
Service Health Check	GET /health	HTTP 200, status: ok	HTTP 200, model_loaded: true (PASSED)
Valid Inference Request	POST /predict (area=1800, beds=3)	HTTP 200, price estimate	HTTP 200, $423,884.13 (PASSED)
Missing Required Field	POST /predict (missing distance_km)	HTTP 422 Unprocessable Entity	HTTP 422, 'Field required' (PASSED)
Incorrect Data Type	POST /predict (area='one thousand')	HTTP 422 Unprocessable Entity	HTTP 422, 'float_parsing' (PASSED)
Invalid Business Value	POST /predict (area=-500, beds=0)	HTTP 422 Unprocessable Entity	HTTP 422, 'greater_than' (PASSED)

5. Docker Containerization & Multi-Service Design (Part C)

The application is packaged into a container using a minimal python:3.11-slim base image. A strict .dockerignore file prevents large raw datasets, virtual environments, and caches from bloating the build context. The Dockerfile exposes port 8000 and runs Uvicorn binding to 0.0.0.0. Furthermore, a multi-service docker-compose.yml connects the FastAPI inference service to an MLflow Tracking Server over an isolated bridge network with volume-persisted backend storage.

6. Local vs. Container Prediction Equivalence (Part D)

To verify complete parity between the local Python environment and the Docker containerized API, five distinct test cases were evaluated simultaneously against both runtimes. The absolute difference was recorded to verify numerical consistency under documented tolerance (|diff| < 0.01).

Case ID	Local Model ($)	Docker Container ($)	Absolute Diff ($)	Parity Status
Case 1	$251,052.91	$251,052.91	$0.0000	EXACT MATCH
Case 2	$423,884.13	$423,884.13	$0.0000	EXACT MATCH

Case 3	$553,672.70	$553,672.70	$0.0000	EXACT MATCH
Case 4	$217,335.68	$217,335.68	$0.0000	EXACT MATCH
Case 5	$711,693.19	$711,693.19	$0.0000	EXACT MATCH

7. Challenges Encountered & Resolutions

• **1. Preprocessing Data Leakage:** Fitting a standard scaler on the full dataset before splitting leaks test set statistics. *Resolution:* Data splitting was moved strictly to src/data.py before scaling, and StandardScaler was embedded into a unified Pipeline fitted solely on X_train.

• **2. DataFrame Column Order in Inference:** When JSON payloads are deserialized, dictionary keys might not preserve the exact column ordering expected by Scikit-learn. *Resolution:* src/api.py and src/predict.py explicitly project inputs into df[EXPECTED_FEATURES] before calling the pipeline.

• **3. Pydantic Type Coercion & Constraints:** Default schemas allow negative values or 0 bedrooms which violate domain logic. *Resolution:* Implemented strict pydantic.Field(..., gt=0, ge=0) constraints to guarantee semantic validity.

• **4. Docker Port Mapping & Host Binding:** Uvicorn defaulting to 127.0.0.1 inside a container is inaccessible to the host machine. *Resolution:* Configured Uvicorn to bind to 0.0.0.0 inside the Dockerfile while mapping host port 8000 to container port 8000.

8. Conclusion & Reflection

The house-price prediction project has been successfully converted from monolithic scripts into a modular, testable, and production-ready MLOps application. All requirements across Sessions 7, 8, and 9 were fulfilled, including modular pipeline execution, unified Scikit-learn pipeline serialization, automated pytest suites, FastAPI REST endpoints with schema enforcement, Docker containerization, and local vs container equivalence verification.